

Taller de Arduino





¿Qué es Arduino?

- Una empresa, una placa, una IDE, un lenguaje de programación
- Plataforma de electrónica open-source





ARDUINO
PROJECT HUB



AUTODESK
Instructables



hackster.io

AN AVNET COMMUNITY





{e.pli}

Grupo de Educación en Programación,
Lenguajes e Información

[https://epli.dc.uba.a](https://epli.dc.uba.ar/)
[r/](https://epli.dc.uba.ar/)



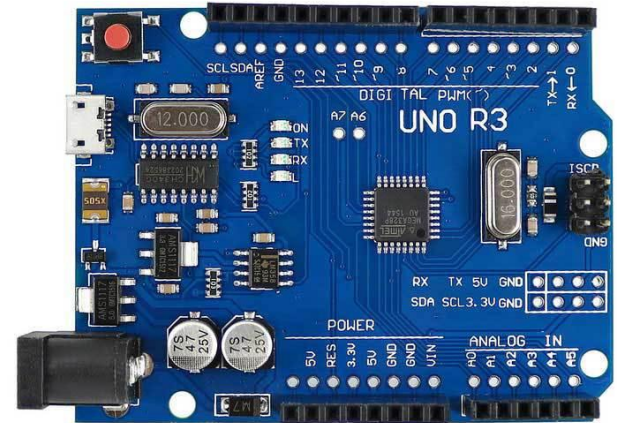


Microcontroladores

- Circuito integrado programable que contiene un CPU, memoria y periféricos de entrada y salida
- Típicamente RISC de 8 o 16 bits con arquitectura Harvard
- Están pensados para realizar tareas puntuales, en vez de ser de propósito general
- No tienen sistema operativo
- Un único programa ejecutándose en *loop*

Arduino UNO R3

- Microcontrolador ATmega328P
- RISC de 8 bits
- Arquitectura Harvard [*]
- 32 KB Flash (donde se guarda el programa)
- 2 KB SRAM (donde se guardan los datos temporales)



[*] Este tema se introduce en las materias del área de Sistemas. Pueden leer más sobre esto (aplicado a Arduino) acá: <https://docs.arduino.cc/learn/programming/memory-guide/>

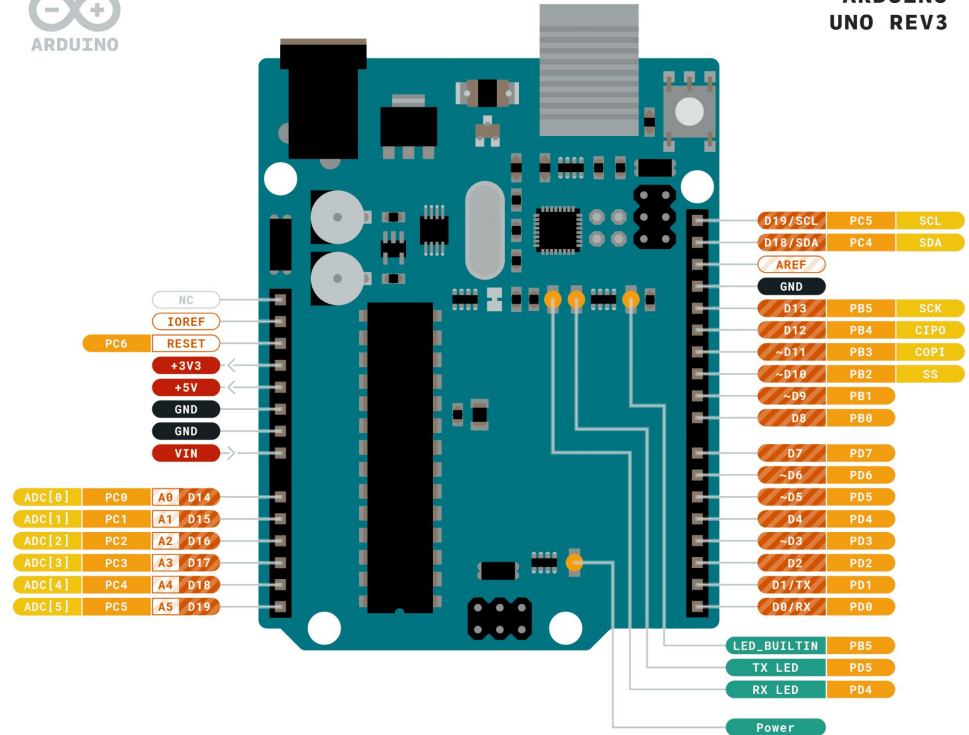




ARDUINO
UNO REV3

Pines - UNO R3

- Conectaremos sensores y actuadores con la placa
- Digitales/analógicos: los usaremos para enviar y recibir datos
- También hay pines para administrar energía o soportar protocolos de comunicación



Ground	Internal Pin	Digital Pin	Microcontroller's Port
Power	SWD Pin	Analog Pin	
LED	Other Pin	Default	

ARDUINO.CC



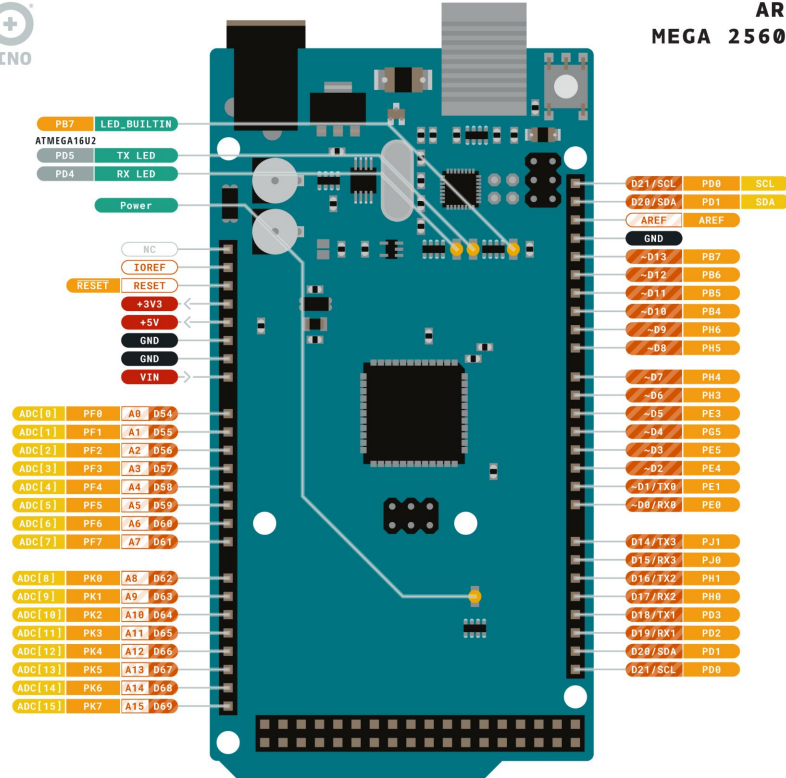
This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, PO Box 1868, Mountain View, CA 94039, USA.



ARDUINO MEGA 2560 REV3

Pines - MEGA 2560

- Conectaremos sensores y actuadores con la placa
- Digitales/analógicos: los usaremos para enviar y recibir datos
- También hay pines para administrar energía o soportar protocolos de comunicación



Ground	Internal Pin	Digital Pin	Microcontroller's Port
Power	SWD Pin	Analog Pin	
LED	Other Pin	Default	

ARDUINO.CC



This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, PO Box 1868, Mountain View, CA 94039, USA.



¿Cómo lo programamos?

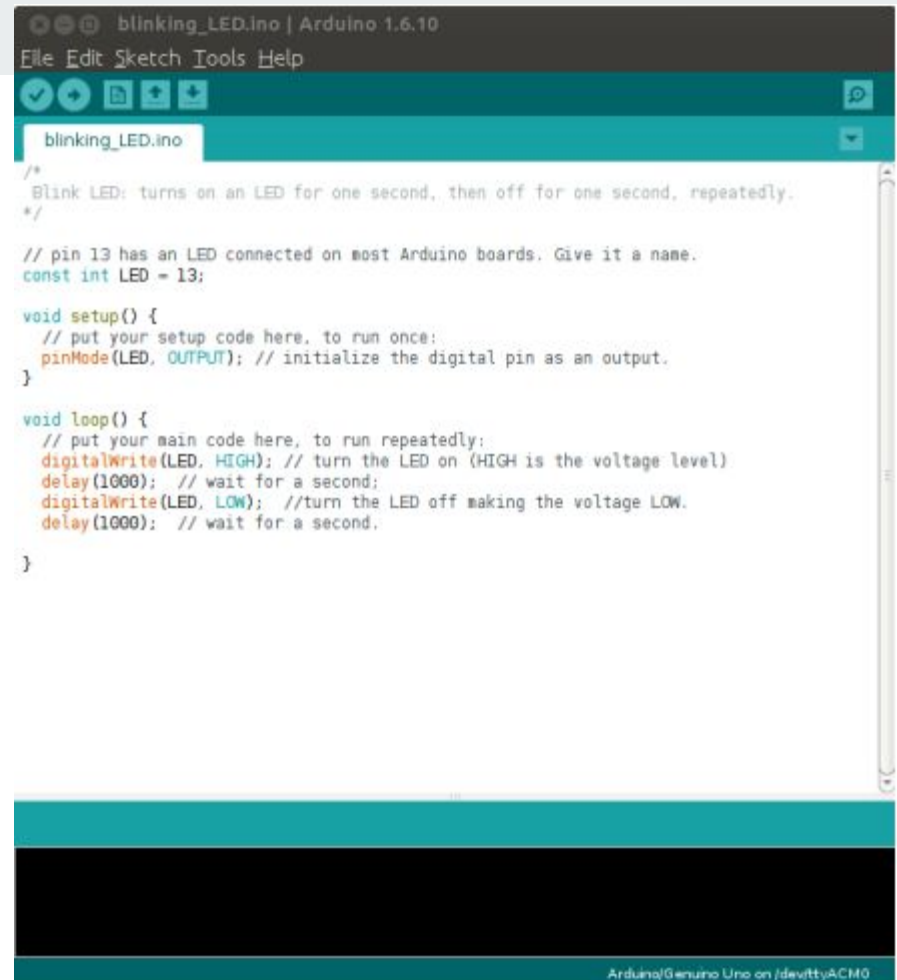
- *sketch* = programa Arduino
- lenguaje Arduino = framework de C++ “simplificado”
- Hay muchas librerías útiles para trabajar con módulos puntuales
- Existen herramientas de programación en bloques desarrolladas para trabajar con Arduino [*]

[*] En el Grupo de Educación en Programación, Lenguajes e Información del DC se desarrolla activamente la herramienta *Arduino en la Escuela*, utilizada en talleres y colegios. Demo disponible acá: <https://aele.dc.uba.ar/home/index.html>

Arduino IDE

- Maneja la compilación, librerías, comunicaciones y carga el código a la placa
- Disponible para descargar acá:

<https://www.arduino.cc/en/software/>



The screenshot displays the Arduino IDE environment. At the top, the title bar reads "blinking_LED.ino | Arduino 1.6.10". Below this is a menu bar with "File", "Edit", "Sketch", "Tools", and "Help". A toolbar with icons for opening, saving, and running is visible. The main text area contains the following C++ code for a blinking LED:

```
/*
 * Blink LED: turns on an LED for one second, then off for one second, repeatedly.
 */

// pin 13 has an LED connected on most Arduino boards. Give it a name.
const int LED = 13;

void setup() {
  // put your setup code here, to run once:
  pinMode(LED, OUTPUT); // initialize the digital pin as an output.
}

void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(LED, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000); // wait for a second;
  digitalWrite(LED, LOW); // turn the LED off making the voltage LOW.
  delay(1000); // wait for a second.
}
```

At the bottom of the IDE window, a status bar indicates "Arduino/Genuino Uno on IDEv1.6.10".

```
chmod +x ./nombreDeImagen
```

```
./nombreDeImagen --no-sandbox
```

Simulador



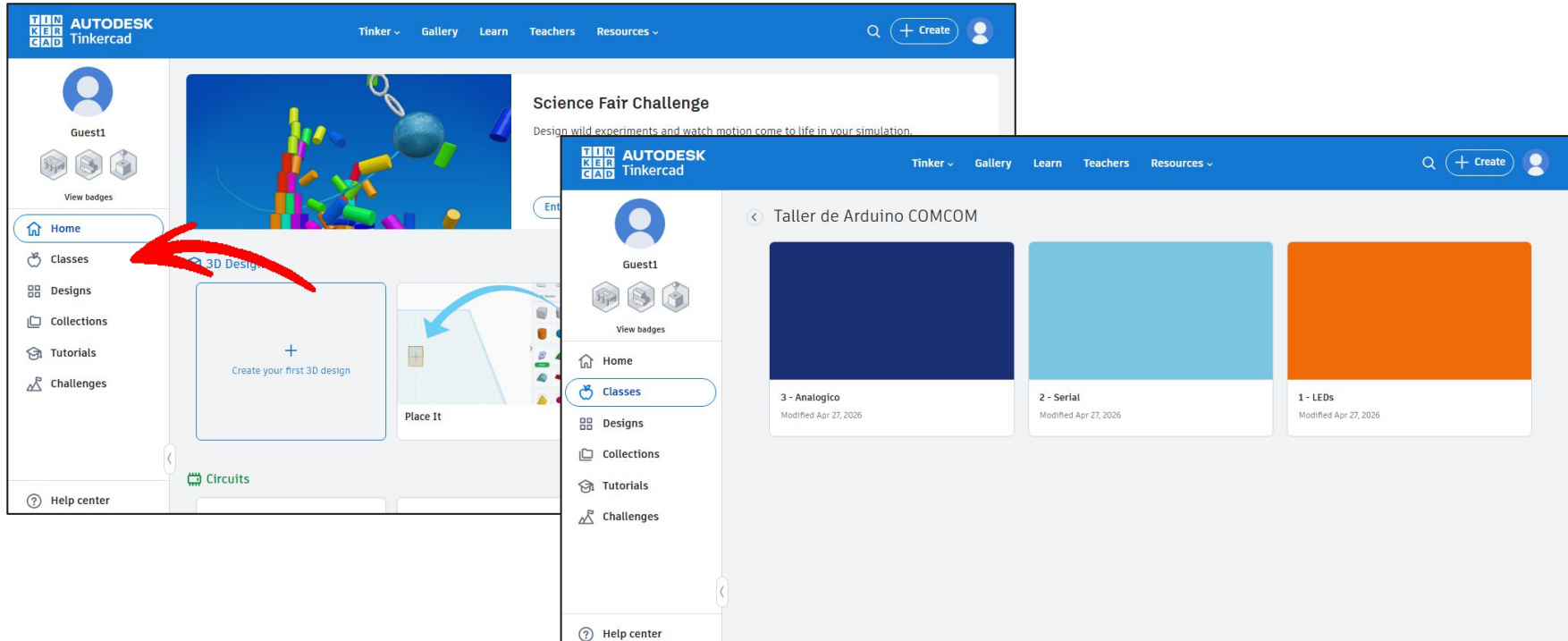
El login code es el nombre de usuario del mail que usaron para inscribirse

Join with Login code

usuario@(gmail.com|dc.uba.ar|yahoo.com|...)

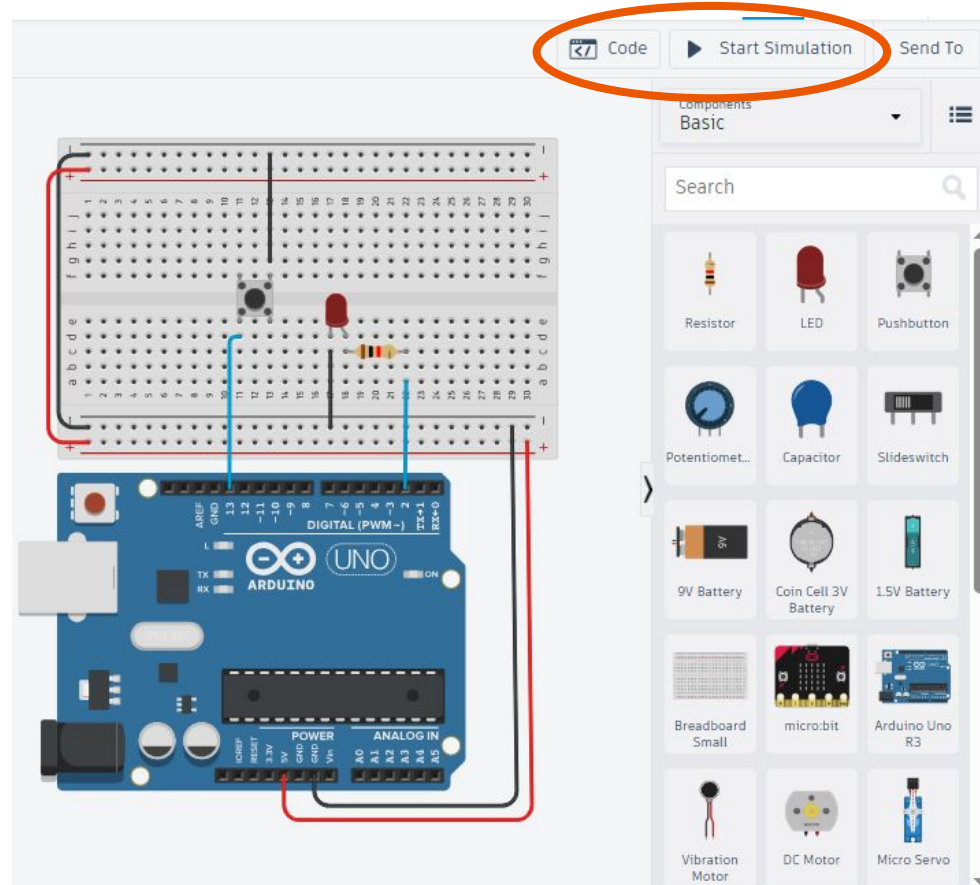
<https://www.tinkercad.com/joinclass/AWPX2FD3G>

Simulador



Simulador

- Circuitos con código y componentes para que puedan experimentar y aprender:
 - sintaxis del lenguaje de programación de Arduino
 - conexión de cada componente que usemos



IMPORTANTE

**DESCONECTAR LA PLACA DE
LA COMPUTADORA CUANDO
ENCHUFAN/DESENCHUFAN
— COSAS**

Escribamos nuestro primer programa

```
void setup() {  
  // Esto se ejecuta una única vez en la vida del Arduino.  
  int a = 2 + 2;  
}  
  
void loop() {  
  // Esto se ejecuta siempre, inmediatamente después de `setup`.  
  int b = 1 + 1;  
}
```

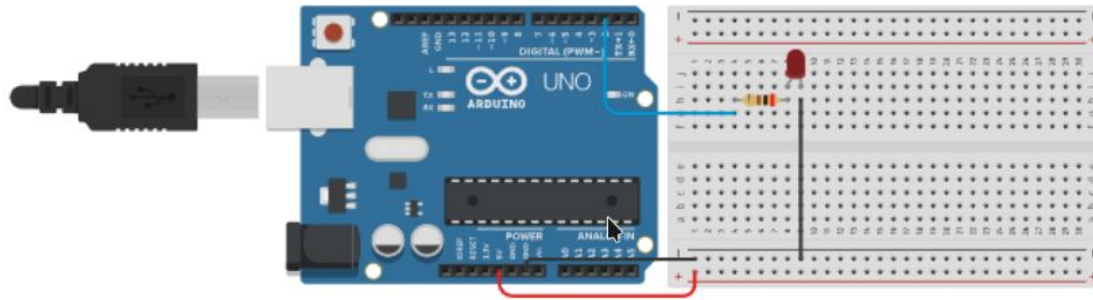
Diferencias entre Python y C (código Python)

```
def foo(parámetro: bool) → None:  
    a: int = 2 + 2  
    b: str = "hola, mundo"  
    if (parámetro):  
        print("parámetro es true!")
```

Diferencias entre Python y C (código C)

```
void foo(bool parámetro) {  
    int a    = 2 + 2;  
    char b[] = "hola, mundo";  
    if (parámetro) {  
        Serial.println("parámetro es true!");  
    }  
}
```

Ejercicio: Hola, mundo



cátodo (-)

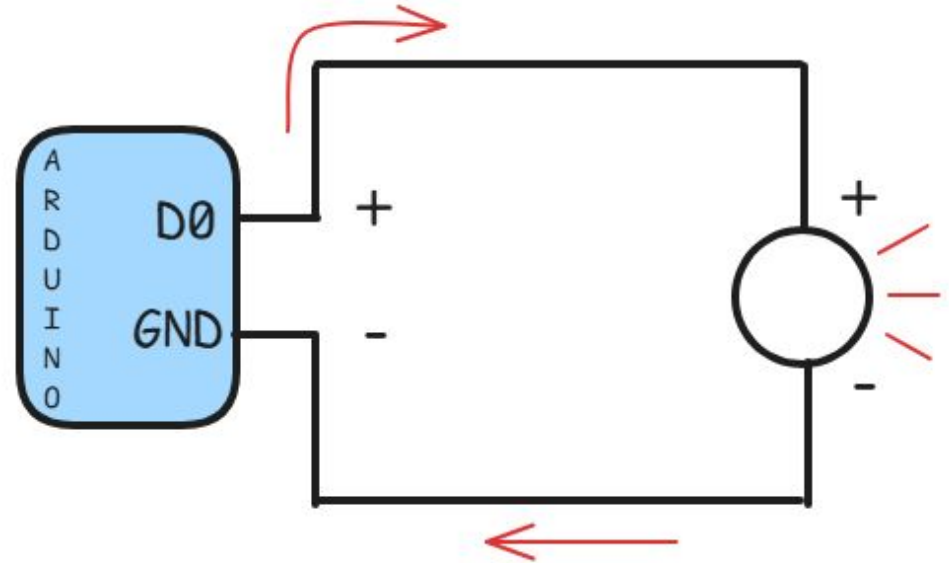
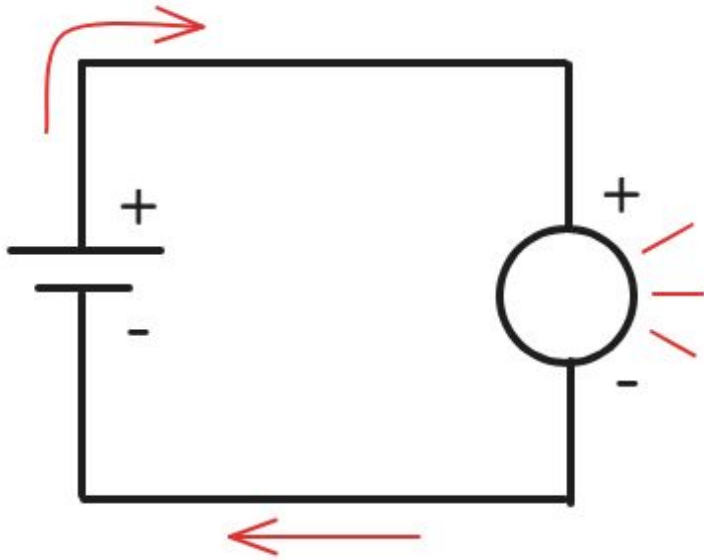
ánodo (+)



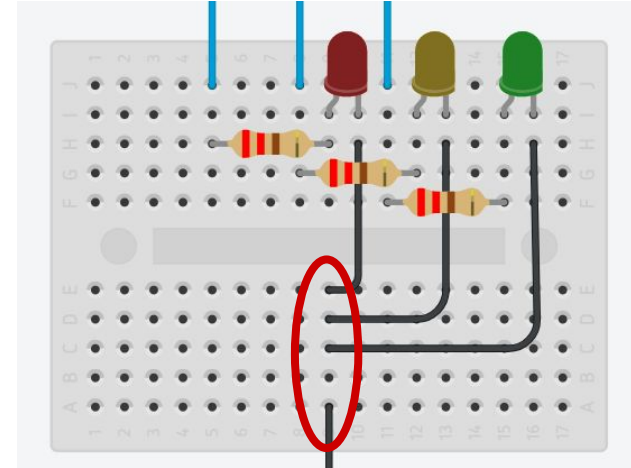
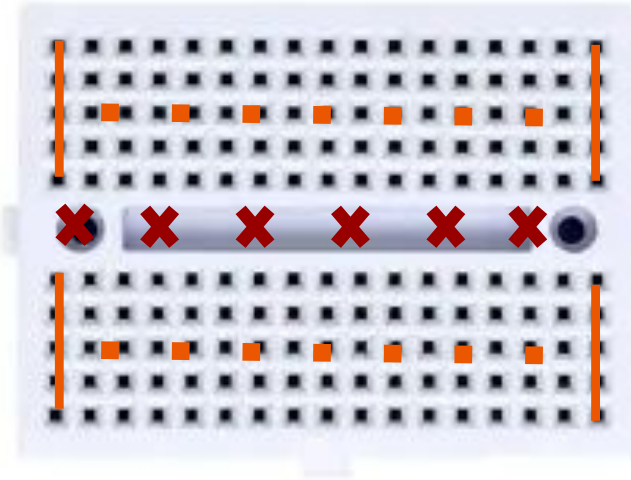
Ejercicio: Hola, mundo

- Lograr encender la luz LED, usando la función *digitalWrite* y la función *pinMode*.
- La función *digitalWrite* toma como parámetro un valor numérico, donde el valor indica el número de un pin digital, y otro parámetro numérico que puede ser 0 (*LOW*) o 1 (*HIGH*) para expresar si queremos o no enviar corriente por ese pin.
- La función *pinMode* toma como parámetro un valor numérico, donde el valor indica el número de un pin, y otro parámetro numérico que puede ser *INPUT* o *OUTPUT* para expresar si ese pin es un pin de entrada, o de salida. En este caso, sería un pin de salida.

Electrónica 101 - Fuentes de alimentación



Electrónica 101 - Breadboards





Ejercicio: Parpadeo con delay

- Logren hacer parpadear la luz LED encendiéndola, luego usando la función *delay* para esperar una cantidad de segundos, y luego apagándola de nuevo, repitiendo este ciclo infinitamente.
- La función *delay* toma como parámetro un valor numérico, expresado en milisegundos, para demorar la ejecución de la próxima instrucción.

Ejercicio: Semáforo

- Logren recrear un semáforo de la vida real siguiendo este patrón:
 - La luz R está encendida. La luz A y V están apagadas.
 - La luz R y la luz A están encendidas. La luz V está apagada.
 - La luz R y la luz A están apagadas. La luz V está encendida.
- Si lo resolvieron con *delay*, intenten modificar el programa para utilizar la función *millis*, la cual devuelve el tiempo en milisegundos desde que el Arduino se encendió.



Ejercicio: Interruptor

- Logren encender o apagar una luz LED utilizando un botón.
- Utilicen la función *pinMode* para configurar el pin de botón como un pin *INPUT* y *digitalRead* para leer el estado del botón.
- Pueden utilizar la función *Serial.println* para enviar por el monitor serial el estado del botón, o cualquier variable de interés para *debuggear* el código.



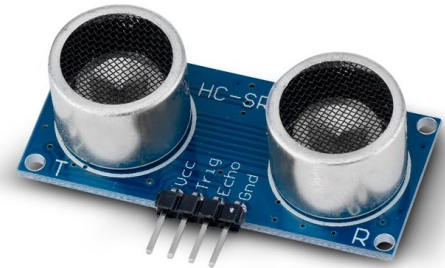
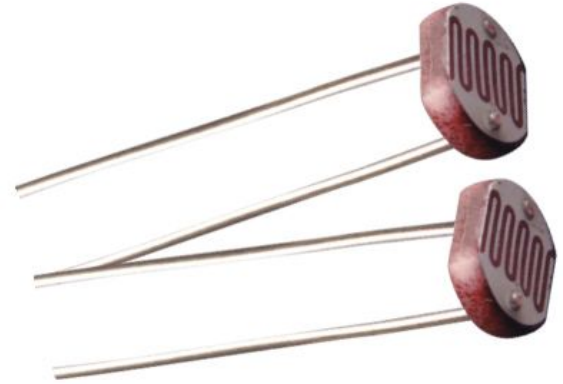
Señales analógicas y digitales

- Digitales: HIGH-LOW, 1-0, 5V-0V
- Analógicas: puede estar en el medio
- Conversor analógico-digital (ADC):
convierte una señal analógica
“clasificándola” en un valor entre 0 y 1023
(por qué?)



Señales analógicas y digitales

- El conversor analógico-digital (ADC) del Arduino permite mapear un voltaje en una representación de 10 bits.
- Cada bit puede tener 2 estados: 0 (*LOW*) o 1 (*HIGH*), con lo cual tenemos 2^{10} posibles valores: un rango 0-1023.
- Esto es útil, ya que no todos los tipos de información que procesaremos del mundo real pueden ser representados sólo de forma binaria



Ejercicio: Joystick

- Logren leer los datos de entrada del joystick.
- Utilicen la función *pinMode* para configurar los 3 pines de entrada del joystick y *digitalRead* para leer el estado del botón interno del joystick, y *analogRead* para leer el estado de los dos ejes analógicos.



Bibliotecas: Servomotores

Podemos controlar un motor con señales digitales de arduino.

La biblioteca [Servo.h](#) permite escribir directamente al pin de Control el ángulo al que queremos que gire el motor: desde 0° a 180°





Ejercicio: Servomotor

- Jugar con el servomotor. Observar qué sucede al escribir distintos valores con la función *write*.
- Hacer que el servomotor gire constantemente de 0° a 180° y viceversa
- Recuerden importar la librería con *#include <Servo.h>* e inicializar un objeto de tipo *Servo* para asociarle en el *setup* su pin de control.

MOMENTO PARO

Form de encuesta



forms.gle/KS2s3uTZALASSKpe6